DAS 839 – NoSQL Systems, Assignment II

MapReduce: Word Association, Document Indexing, Query Retrieval and Scalability

KM Yashaswini BT2024224 (Person 1) Vishal Reddy Kondakindi BT2024102 (Person 2)
Shrey Modi BT2024125 (Person 3) Abhinav Ravi BT2024239 (Person 4)

Repository: `https://github.com/yashaswini450/NoSQL-2`

Abstract

All parts were implemented on Apache Hadoop 3.4.3 and run on the full corpus `Wikipedia-EN-20120601_ARTICLES` (10,000 articles). Problem 1: top-50 frequent words, Pairs and Stripes word association ($d = 2$), and a scalability study of four configurations on nested subsets (10–100%). Problem 2: stemmed document frequency, top-100 inverted index, and TF–IDF query retrieval. All numbers come from our own runs; logs and outputs are in the repository.

1 Environment and setup

1.1 Software and hardware

Everything ran in standalone (local) Hadoop mode: LocalJobRunner on the local filesystem, no HDFS and no YARN. The jobs were run on three different laptops (each teammate ran their own part), so the hardware is listed per part. Timings are therefore only comparable *within* one experiment.

	Problem 1(d) benchmark (Person 2)	Problem 2(a) (Person 3)	Problem 2(b) (Person 4)
OS	Windows 11, WSL2 Ubuntu, Linux 6.6.87.2	Windows 11, WSL2 Ubuntu, Linux 6.18.54.1	Windows 11, WSL2 Ubuntu, Linux 6.18.33.2
CPU	AMD Ryzen 7 7735HS, 16 logical CPUs	AMD Ryzen 9 6900HS, 16 logical CPUs	Intel Core i5-12450H, 12 logical CPUs
RAM seen by WSL	7.4 GiB	7.4 GiB	about 7 GiB
Java	Corretto OpenJDK 11.0.33	Temurin OpenJDK 11.0.33	OpenJDK 11.0.32
Hadoop	3.4.3 standalone	3.4.3 standalone	3.4.3 standalone
JVM heap	`-Xmx1g`	default	default

Table 1: Execution environments. **[Person 1: add the machine used for the original 1(a)/1(b) run; Person 2 reproduced 1(a)/1(b) on the machine in column 1 and all benchmark numbers come from it.]**

Other libraries: `opennlp-tools-1.9.3.jar` (`PorterStemmer`, Problem 2), Python 3 with Matplotlib (plotting) and standard-library scripts for the independent checks.

1.2 Dataset preparation

- `Wikipedia-EN-20120601_ARTICLES.tar.gz` is, despite its name, a plain tar archive (`tar -xf`). It contains **10,000** articles, one file `<id>.txt` each, 166,222,646 bytes. Each article is one line. The test set `Wikipedia-50-ARTICLES.tar` (50 articles, 717,789 bytes) was used for development only.
- `stopwords.txt` has 355 non-empty entries (47 contain an apostrophe and can never match a token after splitting on non-alphanumerics; we kept the official list unchanged).
- **Document ID** = article file name without `.txt`, a stable integer.

1.3 Nested deterministic subsets (Problem 1(d))

`prepare_subsets.sh` sorts the `.txt` file names by byte order and takes the first $\lfloor N \cdot p/100 \rfloor$ files for $p \in \{10, 20, 40, 80, 100\}$, so $S_{10} \subset S_{20} \subset S_{40} \subset S_{80} \subset S_{100}$. The selected articles are written, in order, into about 16 MiB text shards (to avoid one map task per small file); a newline is added where needed so that no co-occurrence window crosses an article boundary.

Subset	Documents	Source bytes	Hadoop input bytes
10%	1,000	11,021,771	11,022,771
20%	2,000	27,653,453	27,655,453
40%	4,000	64,709,490	64,713,490
80%	8,000	130,964,811	130,972,811
100%	10,000	166,222,646	166,232,646

Table 2: Subsets (`subsets.tsv`); the difference is the added newlines.

2 Design and implementation

2.1 Common conventions

All problems use the same text pipeline. Case normalisation: lowercase. Tokenisation: split on `[^a-z0-9]+` (punctuation, apostrophes and non-ASCII letters are separators); empty tokens are dropped. Stop words are read from `stopwords.txt` shipped through Hadoop's DistributedCache and loaded into a `HashSet` in `Mapper.setup()`; a token is skipped if it is in the set (this replaces the substring `replaceAll` of the starter `WordCount2.java`, which corrupts words). Problem 2 additionally applies the OpenNLP `PorterStemmer` *after* stop-word removal (`TextPreprocessor.terms()`, shared by 2(a) and 2(b)). Ties are always broken by ascending lexicographic order of the word/term (numeric order of DOC_ID for documents).

2.2 Problem 1(a): top-50 frequent words (`Top50Words.java`)

Map emits $(w, 1)$ per non-stop token; a combiner sums per split; a single reducer accumulates corpus totals and, in `cleanup()`, sorts by (frequency desc, word asc) and writes the first 50 as `WORD<TAB>FREQUENCY`.

```
setup():   stopwords <- cached stopwords.txt
map(k, line):  for token in line.toLowerCase().split("[^a-z0-9]+"):
                   if token != "" and token not in stopwords: emit(token, 1)
combine/reduce(w, counts): total[w] <- sum(counts)
cleanup(): sort total by (count desc, w asc); emit first 50
```

2.3 Problem 1(b): Pairs (`WordAssociationPairs.java`)

The vocabulary V_{50} (`top50_words.tsv`) is loaded from the cache. For each line the filtered (non-stop) token stream $t_0, \ldots, t_{n-1}$ is scanned; for every position i with $t_i \in V_{50}$, each neighbour t_j with $1 \le |i - j| \le d = 2$ and $t_j \in V_{50}$, $t_j \neq t_i$, emits the *ordered* pair $(u, v) = (t_i, t_j)$ with count 1. Non-vocabulary tokens keep their positions (the window counts filtered tokens, not only vocabulary words). Pairs are **ordered**, $C(u, v) = C(v, u)$ holds by symmetry of the window, self-pairs are excluded from counts and denominators. The reducer sums $C(u, v)$, computes $\sum_w C(u, w)$ and $P(v \mid u) = C(u, v) / \sum_w C(u, w)$, and keeps the 5 largest $C(u, v)$ per u (ties by v ascending). Local-aggregation modes: `false`/none (emit every co-occurrence), `function` (aggregate in a per-record `HashMap` flushed at the end of `map()`), and `true`/class (in-mapper `HashMap` across the whole split, flushed once in `cleanup()` and cleared, plus a `PairsCombiner`).

2.4 Problem 1(c): Stripes (`WordAssociationStripes.java`)

Same tokenisation, window, vocabulary and ordered-pair convention as Pairs. For each focal occurrence $u \in V_{50}$ the mapper builds a stripe, a map $v \mapsto$ count of its neighbours in the window. Without aggregation each non-empty stripe is emitted immediately as (u, stripe) (`MapWritable`); with aggregation one stripe per word is kept for the whole split and emitted in `cleanup()` (state is bounded by $|V_{50}| = 50$). The reducer merges stripes, computes the denominator and $P(v \mid u)$ exactly as in Pairs and writes the same schema (`u<TAB>v<TAB>C(u,v)<TAB>P(v|u)`, six decimals).

2.5 Problem 1(d): local aggregation and scalability

`run_scalability.sh` runs Pairs and Stripes, each with aggregation off and on (four configurations), one job at a time, on the five nested subsets with $d = 2$ and the same top-50 vocabulary. 10%, 40% and 100% are run three times, 20% and 80% once. Wall-clock time is measured from just before the job launch until the process exits (JVM startup, job, and output writing; not extraction, subset preparation or compilation). Mapper-output records and bytes are read from the Hadoop counters. `plot_scalability.py` draws input size vs. runtime.

2.6 Problem 2(a): DF and inverted index (`DocumentFrequency.java`, `InvertedIndex.java`)

A custom `WholeDocumentInputFormat` (a `CombineFileInputFormat`) returns one record per article, so each `map()` call sees exactly one document, and files are combined into splits of at most 64 MB (3 map tasks instead of 10,000).

```
Job 1 (DF):   map(docId, text): for t in set(terms(text)): emit(t, 1)     # once per doc
              combine = reduce: emit(t, sum)                               # = DF(t)
Job 2 (top100): each mapper keeps a TreeSet ordered by (df desc, term asc), size 100,
              emits it in cleanup(); one reducer merges to the global top 100.
InvertedIndex: map(docId, text): tf <- count of each top-100 term in text;
                                 for (t,c) in tf: emit(t, docId ":" c)
               reduce(t, postings): sort by numeric docId; emit(t, "d1:tf1,d2:tf2,...")
```

DF counts *distinct documents* (a term used 50 times in one article counts once). Because every combiner value is one distinct (term, document) pair, summing early cannot double-count. Top-100 ties: DF descending, then TERM ascending. Output schemas: `TERM<TAB>DF`; `TERM<TAB>DOC_ID:TF,DOC_ID:TF,...`.

2.7 Problem 2(b): query retrieval (`QueryRetrieval.java`)

Inputs. `inverted_index.tsv` (MapReduce input), `top100_df.tsv` (DistributedCache; source of DF), N (from `doc_count.txt`), `stopwords.txt`, and the query string.

Query handling (driver).

1. The query goes through the *same* pipeline as the documents (`TextPreprocessor.terms()`).
2. **Repeated query terms** collapse into the *set* Q (matching $t \in Q \cap D$); the score is not multiplied by query multiplicity.
3. Stop words disappear in step 1. Terms **absent from the index** (not among the 100 indexed terms) are logged as ignored and skipped.
4. If no usable term remains (e.g. only stop words), the driver prints "no documents can match", runs no job and writes no output. If usable terms exist but no document contains them, the output is empty.

Scoring. $\mathrm{IDF}(t) = \ln\big(N/\mathrm{DF}(t) + 1\big)$ (natural log), $S(D,Q) = \sum_{t \in Q \cap D} \mathrm{TF}(t,D) \cdot \mathrm{IDF}(t)$. The formula is used unchanged (no length normalisation).

```
driver:  Q <- set(terms(query)) intersect indexedTerms(top100_df.tsv); if Q empty: report, exit
         conf: Q, N; cacheFile: top100_df.tsv; 1 reducer
setup(): dfByTerm <- cached top100_df.tsv
map(line "TERM \t d1:tf1,d2:tf2,..."):
    if TERM not in Q: return
    idf <- ln(N / dfByTerm[TERM] + 1)
    for each posting d:tf: emit(d, TERM ":" tf*idf)
    if #postings != dfByTerm[TERM]: counter DF_MISMATCH++       # consistency with 2(a)
reduce(doc, values):  perTerm <- TreeMap(term -> contribution)  # summed in term order
    hit <- (sum of contributions, sorted matched terms)
cleanup():  sort hits by score desc, then DOC_ID asc (numeric); write top 10:
            RANK \t DOC_ID \t MATCHED_TERMS \t SCORE (6 decimals)
```

Design decisions. (i) DF is read from Part 2(a)'s `top100_df.tsv`, as the assignment requires, and cross-checked against the posting count. (ii) Tie-breaking: equal scores are ordered by DOC_ID ascending, *compared numerically* (so 999 precedes 1000); contributions are summed in sorted-term order, so floating-point sums do not depend on shuffle order. (iii) One reducer gives a global top-10; at most N candidate documents fit in memory.

3 Correctness checks

3.1 Problem 1(a): top-50 words

All 50 words in `top50_words.tsv` were checked against `stopwords.txt`: **0** stop words appear. The ranking uses (frequency desc, word asc), so ties are deterministic. Person 2 reproduced the same 50 words and frequencies by re-running `Top50Words` on the prepared corpus, and an independent Python recount from the raw article files agreed.

#	word	frequency	#	word	frequency
1	apos	273,057	6	2	66,358
2	quot	267,968	7	category	60,534
3	s	207,334	8	this	58,764
4	http	79,108	9	0	54,719
5	1	77,032	10	www	54,136

Table 3: First ten rows of `top50_words.tsv`. Words such as `apos`, `quot` (HTML entities), `http`, `www` and bare numbers appear because the agreed tokenizer does not filter them (see Section 6).

3.2 Problems 1(b) and 1(c): Pairs vs. Stripes and aggregation modes

- The three Pairs modes (none / function / class) produce identical `pairs_top5.tsv` (`diff -u` clean, 250 rows = 5 associations × 50 words).
- `diff pairs_top5.tsv stripes_top5.tsv` is empty: **Stripes reproduces Pairs exactly** (counts and probabilities). This was re-checked on the merged `main`.
- Within the benchmark, all four configurations produced identical output at every subset size and trial, and an independent recount from the original articles matched at all five sizes.
- Symmetry sanity check: $C(\text{united}, \text{states}) = C(\text{states}, \text{united}) = 21{,}734$, yet $P(\text{states} \mid \text{united}) = 0.664059 \neq P(\text{united} \mid \text{states}) = 0.685507$ because the denominators differ.

u	v	$C(u, v)$	$P(v \mid u)$
united	states	21734	0.664059
united	category	2475	0.075621
united	county	946	0.028904
united	quot	804	0.024565
united	1	688	0.021021

Table 4: Top-5 associations of "united" (rows of `pairs_top5.tsv`).

3.3 Problem 2(a): DF and inverted index

`check_problem2a.py` (plain Python, independent of the Java code and of the stemmer) reports **ALL CHECKS PASSED** on the full corpus: $N = 10{,}000$ equals the number of files; `df_all.tsv` (524,390 terms) has no duplicate term and $1 \leq \text{DF} \leq N$; `top100_df.tsv` equals the true top 100 of `df_all.tsv` under (DF desc, TERM asc); the boundary DF 4832 at rank 100 is held by a single term (no tie excluded); the index holds exactly the 100 terms; **number of postings equals DF for every term**, so no document is counted twice; no duplicate DOC_ID inside a posting list; every TF≥ 1; postings sorted. A raw-text cross-check for 12 terms unchanged by stemming (e.g. `american`, `first`, `form`) found every article containing the raw token in the postings with TF at least the raw count. Hand checks with `grep` on the 50-article set: `includ` DF 42 (grep 42), `peopl` DF 36 (grep 36), TF of `includ` in doc 1338 = 12 (grep 12). Stop words cannot appear: they are removed before stemming and no top-100 term is spelled like a stop word.

rank	term	DF	rank	term	DF
1	s	9836	6	other	8415
2	apo	9518	7	1	8350
3	categori	9381	8	link	8295
4	refer	9340	9	thi	8190
5	quot	8674	10	2	8105

Table 5: Top-10 terms by DF (full list in `top100_df.tsv`; rank 100 is `major`, DF 4832).

Representative inverted-index records (first 10 postings; `s 698:39` means "term `s` occurs 39 times in article 698"):

```
s      698:39,700:95,708:2,709:8,740:10,842:7,857:8,864:148,900:44,902:91
major  698:8,700:3,709:2,740:2,842:1,864:2,902:2,1178:1,1198:3,1214:4
```

3.4 Problem 2(b): query ranking

Query used. No instructor-supplied query had been announced, so we used our own multi-term query **"history united states"**. It becomes $Q = \{\text{histori}, \text{state}, \text{unit}\}$ (none of the three words is a stop word). $N = 10{,}000$.

The program accepts any query; another query needs only a re-run of `run_query.sh`.

Term t	DF(t) (`top100_df.tsv`)	IDF(t) = ln(10000/DF + 1)
histori	6657	0.917162
state	7299	0.862911
unit	6498	0.931745

9,018 documents contain at least one query term. Output of the Hadoop job (`query_top10.tsv`):

RANK	DOC_ID	MATCHED_TERMS	SCORE
1	334701	state,unit	580.808168
2	462443	histori,state,unit	384.904727
3	317144	state,unit	298.119407
4	31655	histori,state,unit	241.365556
5	18858	histori,state,unit	236.636569
6	201936	histori,state,unit	236.551865
7	62513	histori,state,unit	234.387576
8	21650	histori,state,unit	230.939936
9	560898	state,unit	212.701161
10	225102	histori,state,unit	208.959599

Worked score calculations (TF from `inverted_index.tsv`, IDF from the table above; products rounded, totals in double precision):

$S(334701, Q) = 672 \times 0.862911 + 1 \times 0.931745 = 579.876423 + 0.931745 = \mathbf{580.808168}$

$S(462443, Q) = 1 \times 0.917162 + 215 \times 0.862911 + 213 \times 0.931745 = 0.917162 + 185.525939 + 198.461626 = \mathbf{384.904727}$

$S(317144, Q) = 163 \times 0.862911 + 169 \times 0.931745 = 140.654549 + 157.464858 = \mathbf{298.119407}$

Validation. (i) `verify_retrieval.py`, a Hadoop-free re-implementation that reads DF from `top100_df.tsv`, gives output *identical* to the Hadoop job on the full corpus (and on the 50-article set, $N = 50$, top document 1338 with score 46.733929). (ii) The counter `DF_MISMATCH` never fired: the posting count equals DF for every queried term. (iii) Edge cases that were run: a stop-word-only query (`the of and`) → no terms, message, no job; `zzzqqq history history History` → `zzzqqq` ignored as absent, repeated `history` collapsed, ranking by `histori` only; documents 7174 and 45633 have exactly equal scores (72.455768) and are ordered numerically.

Why this ranking is reasonable (and its limits). The formula is applied exactly as specified: raw TF×IDF, no length normalisation. All three query terms are very common (DF 65–73% of the corpus), so their IDFs are low and almost equal (0.86–0.93) and **TF dominates** the score. Hence the top document (334701, a long list of numbered New York *state* highways) does not contain "histori" at all: it uses "state" 672 times. Documents 462443 (a US-Congress table with 215 "state", 213 "unit") and 317144 (an airport list with many "United States" entries) are likewise repetition-driven, while documents matching all three terms fill ranks 4–8 and 10. This is a property of the specified scoring function (long, repetitive documents win; very common terms discriminate poorly), not of the implementation, and we did not change the formula.

4 Performance evaluation

4.1 Problem 1(d): scalability study

Machine: Person 2's laptop (Table 1, column 1), standalone Hadoop, one job at a time, one reducer. Cells for 10%, 40% and 100% are the **median of three runs**; 20% and 80% are single runs (as required). Sizes are the actual Hadoop input bytes.

Subset	Docs	MiB	Pairs-NoAgg (s)	Pairs-Agg (s)	Stripes-NoAgg (s)	Stripes-Agg (s)
10%	1,000	10.51	7.64	7.70	8.70	7.77
20%	2,000	26.37	11.70	10.84	12.95	10.94
40%	4,000	61.72	17.88	16.33	19.77	16.17
80%	8,000	124.91	35.58	28.51	33.71	27.30
100%	10,000	158.53	34.27	30.49	38.44	30.24

Table 6: Wall-clock runtime (seconds). All 44 individual measurements are in `scalability_runs.csv`.

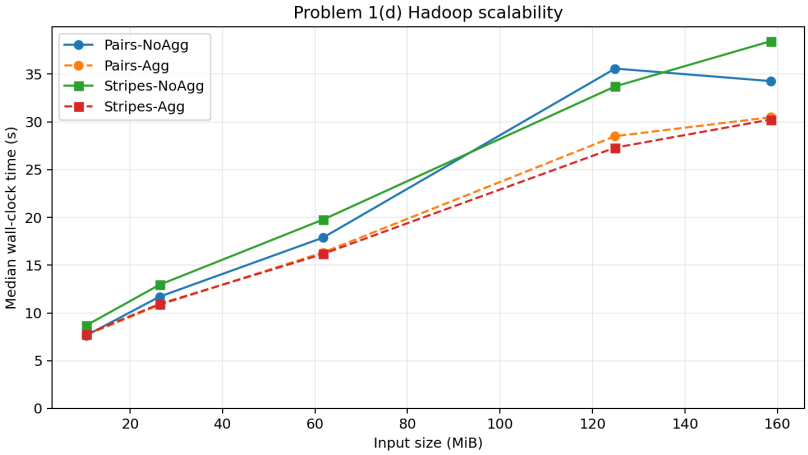


Figure 1: Input size vs. runtime for the four configurations (`scalability.png`).

Configuration	Median map output records 10%	Median map output records 100%	Median map output bytes 10%	Median map output bytes 100%
Pairs-NoAgg	139,578	1,540,824	2,214,954	24,567,766
Pairs-Agg	2,280	23,526	38,020	392,606
Stripes-NoAgg	103,663	1,169,590	2,810,697	31,272,291
Stripes-Agg	50	500	33,159	342,149

Table 7: Intermediate data (Hadoop counters `Map output records/bytes`; values for all five sizes are in `scalability_results.csv`).

Analysis.

- **Trend.** Input grows 15.1× from 10% to 100%, while runtime grows only 3.9×–4.5× across the configurations: runtime is clearly sub-proportional because JVM/Hadoop startup and output writing are a fixed cost that dominates the small runs (about 7.6 s at 10%). The 80% Pairs-NoAgg value (35.58 s) is above its 100% median (34.27 s); the 80% cells are single runs, so this is run-to-run noise on a laptop and not a real trend.
- **Local aggregation helps.** It reduces mapper output by orders of magnitude: Pairs from 1,540,824 to 23,526 records (−98.5%; at most $50 \cdot 49 = 2450$ distinct ordered pairs per mapper and 10 map tasks) and Stripes from 1,169,590 to 500 records, i.e. 50 stripes per mapper (−99.96%, bytes −98.9%). Runtime improves by 11% (Pairs) and 21% (Stripes) at 100%, and Stripes-Agg is the fastest at 30.24 s, just ahead of Pairs-Agg (30.49 s).
- **Pairs vs. Stripes without aggregation.** Stripes emits fewer records (1.17 M vs. 1.54 M) but more bytes (31.3 MB vs. 24.6 MB), since each record carries a map of neighbour counts; it was also slower (38.44 s vs. 34.27 s).
- The Pairs "function" mode (aggregation per record only) was measured by Person 1 on the full corpus: 674,998 map output records versus 1,540,824 without aggregation (−56.2%), i.e. between no aggregation and full in-mapper aggregation.
- The mapper-byte counters count serialised emissions; shuffle bytes differ slightly (see `problem1_hadoop_counters.txt`).

4.2 Problem 2: indexing and retrieval timings

These are single runs in standalone mode (no timing study was required for Problem 2); each includes JVM and job start-up and output writing.

Job	50 articles	10,000 articles
2(a) DF + top-100 (two chained jobs)	3.73 s	20.08 s
2(a) inverted index	2.41 s	11.97 s
2(b) query retrieval (end-to-end, incl. `javac+jar`)	–	12.4 s

Counters (full corpus): DF job: 3 map tasks, 10,000 input records, 7,153,978 map output records, reduced to 524,390 terms; the combiner cut shuffle records from 7.15 M to 0.72 M (−89.9%). Index job: 615,076 postings. Query job: 100 index lines read, 20,454 map output records, 9,018 matching documents, 10 output rows. The 2(b) input is only the 5.4 MB index (100 lines), which is why retrieval is cheap compared with building the index.

5 Reproducibility

5.1 Repository layout

Files	Content
`Top50Words.java`, `top50_words.tsv`	1(a) code and output
`WordAssociationPairs.java`, `pairs_top5.tsv`	1(b) code and output
`WordAssociationStripes.java`, `stripes_top5.tsv`	1(c) code and output
`prepare_subsets.sh`, `run_scalability.sh`, `plot_scalability.py`	1(d) scripts
`scalability_results.csv`, `scalability_runs.csv`, `subsets.tsv`, `scalability.png`	1(d) results
`DocumentFrequency.java`, `InvertedIndex.java`, `TextPreprocessor.java`, `WholeDocumentInputFormat.java`	2(a) code
`results_full/`, `results_50/`	2(a) outputs, logs, checks (full / test set)
`QueryRetrieval.java`, `run_query.sh`, `verify_retrieval.py`	2(b) code, runner, check
`query_top10.tsv`, `results_2b/`	2(b) output, worked scores, run log
`problem_1.txt`, `problem_2.txt`	run instructions (reproduced below)

5.2 Main commands

```
# Problem 1 (full instructions in problem_1.txt below)
hadoop jar problem1.jar Top50Words stopwords.txt <input_dir> out_top50 50
hadoop jar problem1.jar WordAssociationPairs   stopwords.txt top50_words.tsv <input_dir> out_pairs   <false|
    function|true> 2
hadoop jar problem1.jar WordAssociationStripes stopwords.txt top50_words.tsv <input_dir> out_stripes <false|true>
bash prepare_subsets.sh <articles_dir> subsets ; bash run_scalability.sh subsets stopwords.txt top50_words.tsv
    problem1.jar out_bench

# Problem 2
./run_problem2a.sh Wikipedia-EN-20120601_ARTICLES results_full
./run_query.sh results_full "history united states" output_query
python3 verify_retrieval.py results_full "histori,state,unit" --explain 3
```

Fixed configuration: window $d = 2$; one reducer per job; Top-K 50 and 100; split size limit 64 MB for 2(a); `-Xmx1g` for the benchmark.

5.3 Contents of `problem_1.txt` and `problem_2.txt`

problem_1.txt

```
DAS 839 NoSQL Systems, Assignment 2
Problem 1 | Word association and scalability

Environment
-----------
The benchmark used Ubuntu on WSL2 (Linux 6.6.87.2), Corretto OpenJDK 11.0.33
and Apache Hadoop 3.4.3 in standalone mode. WSL saw 16 logical CPUs on an AMD
Ryzen 7 7735HS and 7.4 GiB of RAM. All jobs used a 1 GiB maximum Java heap, the
local filesystem, one reducer and the same Hadoop configuration. Python 3
runs the subset and benchmark scripts. Matplotlib makes the plot.

Place the Person 2 files in the root of NoSQL-2, alongside Top50Words.java,
WordAssociationPairs.java and stopwords.txt from main. The source used in the
benchmark was main commit 8b6e5a4beca1ddf9700d69a6b1302b7a0b236681.
In Ubuntu or WSL, make java, javac and hadoop available on PATH and run these
commands from the repository root. Use a new output_problem1 directory.

    export HADOOP_HEAPSIZE_MAX=1g
    mkdir -p output_problem1/classes
    javac -cp "$(hadoop classpath)" -d output_problem1/classes \
        Top50Words.java WordAssociationPairs.java WordAssociationStripes.java
    jar cf output_problem1/problem1.jar -C output_problem1/classes .
    JAR=output_problem1/problem1.jar

Prepare the data
----------------
Replace the archive path with the location of the official LMS download.

    tar -xf /path/to/Wikipedia-EN-20120601_ARTICLES.tar.gz -C output_problem1
    ARTICLES=output_problem1/Wikipedia-EN-20120601_ARTICLES
    SUBSETS=output_problem1/subsets
    bash prepare_subsets.sh "$ARTICLES" "$SUBSETS"

The script sorts .txt article filenames by byte order and takes the first
floor(N * percent / 100) files. This makes the 10, 20, 40, 80 and 100 percent
subsets nested. It writes the selected articles in order into about 16 MiB
text shards to avoid creating a separate map task for each small article.
It adds a newline where needed to keep the final line of one article separate
from the first line of the next article. No window crosses a line boundary.
subsets.tsv records article counts, original bytes and actual Hadoop input
bytes, including added newlines.
```

```
Problem 1(a) and 1(b)
--------------------
Generate a single vocabulary on the entire corpus, then reuse it for every
run. The repository already contains Yashaswini's full-corpus top50_words.tsv.
To reproduce it, run the first command and use the resulting TSV below.

    hadoop jar "$JAR" Top50Words stopwords.txt "$SUBSETS/pct-100" \
        output_problem1/top50 50
    cp output_problem1/top50/part-r-00000 output_problem1/top50_words.tsv
    TOP50=output_problem1/top50_words.tsv
    diff -u <(tr -d '\r' < top50_words.tsv) "$TOP50"

    hadoop jar "$JAR" WordAssociationPairs stopwords.txt "$TOP50" \
        "$SUBSETS/pct-100" output_problem1/pairs false 2

Problem 1(c)
------------
    hadoop jar "$JAR" WordAssociationStripes stopwords.txt "$TOP50" \
        "$SUBSETS/pct-100" output_problem1/stripes true
    diff -u output_problem1/pairs/part-r-00000 output_problem1/stripes/part-r
       -00000

Pairs and Stripes both lowercase each line, split on non-ASCII-alphanumeric
characters and remove stop words. A window reaches two filtered positions
to either side of a word. Non-vocabulary tokens keep their positions. Each
ordered association has two distinct vocabulary words. This matches Pairs
and excludes self-associations from both counts and denominators.

The Stripes mapper builds a small map of neighbor counts for each focal word
occurrence. With aggregation off it emits each nonempty stripe immediately.
With aggregation on it stores one stripe per word for the whole input split
and writes them in cleanup. Its state is bounded by the 50-word vocabulary.
It emits one stripe per key per mapper, so a combiner would have nothing
additional to merge. Pairs with aggregation uses its in-mapper counts and
Hadoop combiner.
The reducer adds the stripes for each focal word, computes the total over
all neighbors, then emits the five highest counts. Ties sort by neighbor
word. Each TSV row contains focal word, neighbor, count and count divided by
the total, rounded to six decimal places.

Problem 1(d)
------------
    bash run_scalability.sh "$SUBSETS" stopwords.txt "$TOP50" "$JAR" \
        output_problem1/benchmark
    python3 plot_scalability.py output_problem1/benchmark/scalability_results.
       csv \
        output_problem1/benchmark/scalability.png

The script runs Pairs and Stripes with aggregation off and on, one job at a
time. It runs three trials at 10%, 40% and 100%, and one at 20% and 80%.
All runs at a given input size must produce identical TSVs. Every invocation
gets a separate output directory and log. The jobs launch from the benchmark
directory so a local file cannot shadow the provided vocabulary in Hadoop's
distributed cache. The script writes every trial to
scalability_runs.csv, the median by configuration and size to
scalability_results.csv, and the 100% Stripes output to stripes_top5.tsv.

Wall-clock time starts immediately before launching the Hadoop job and ends
when it exits. It includes JVM startup, the Hadoop job and writing output.
Downloading, extracting, making subsets and compiling are outside the timer.
Standalone mode has no HDFS transfer. Map output records and Map output bytes
are the mapper-emission counters. A combiner's effect on shuffled bytes can
differ from its effect on these counters.

The measured jobs used Hadoop's RunJar launcher with Java -Xmx1g and the
3.4.3 client API/runtime jars from share/hadoop/client on the classpath, plus
etc/hadoop and commons-logging, SLF4J and reload4j from share/hadoop/common/lib.
The same launcher and configuration were used for all configurations. The
standard hadoop jar command above runs the same jobs and may have different
launcher overhead. No jobs ran in parallel.
```

problem_2.txt

```
PROBLEM 2 - Document Indexing and Query-Based Retrieval
Run instructions for Part 2(a) (DF + inverted index) and Part 2(b) (query
      retrieval).

================================================================================
ENVIRONMENT
================================================================================
 OS        : Windows 11 + WSL2 Ubuntu (commands below are run inside WSL)
 Java      : OpenJDK 11
 Hadoop    : 3.4.3, standalone (local) mode, LocalJobRunner, local filesystem (
      no HDFS/YARN)
 Libraries : opennlp-tools-1.9.3.jar (PorterStemmer) - keep in the working
      folder, not committed
 Inputs    : stopwords.txt, Wikipedia-EN-20120601_ARTICLES.tar.gz (N = 10000
      articles),
             Wikipedia-50-ARTICLES.tar (test set, N = 50)
 Setup     : source ~/hadoop-env/env.sh   (any shell where `hadoop` and `java`
      work is fine;
             it must export HADOOP_HOME/JAVA_HOME and put `hadoop` on PATH)

Shared preprocessing (identical in 2(a) and 2(b), implemented once in
      TextPreprocessor.java):
   lowercase -> split on [^a-z0-9]+ -> drop empty tokens -> drop stop words (raw
       token,
   before stemming) -> PorterStemmer.stem -> drop empty stems.
Document ID = article file name without ".txt" (stable integer ID).

================================================================================
DATASET PREPARATION
================================================================================
 tar -xf Wikipedia-50-ARTICLES.tar
 tar -xf Wikipedia-EN-20120601_ARTICLES.tar.gz     # despite the .gz name it is
      a plain tar

================================================================================
PART 2(a)  -  DF, Top-100 terms, inverted index
================================================================================
 One-shot script (compiles, runs DF -> Top-100 -> InvertedIndex, copies outputs)
      :
   ./run_problem2a.sh Wikipedia-50-ARTICLES            results_50
   ./run_problem2a.sh Wikipedia-EN-20120601_ARTICLES   results_full
   python3 check_problem2a.py results_full Wikipedia-EN-20120601_ARTICLES
      stopwords.txt

 Equivalent manual commands:
   export HADOOP_CLASSPATH=$PWD/opennlp-tools-1.9.3.jar
   mkdir classes
   javac -classpath "$(hadoop classpath):opennlp-tools-1.9.3.jar" -d classes \
       TextPreprocessor.java WholeDocumentInputFormat.java DocumentFrequency.
      java InvertedIndex.java
   jar cf problem2_index.jar -C classes .
   hadoop jar problem2_index.jar DocumentFrequency -libjars opennlp-tools-1.9.3.
      jar \
       stopwords.txt Wikipedia-EN-20120601_ARTICLES out_df
   hadoop jar problem2_index.jar InvertedIndex -libjars opennlp-tools-1.9.3.jar
      \
       stopwords.txt top100_df.tsv Wikipedia-EN-20120601_ARTICLES out_index

 Outputs (also copied to results_full/ and results_50/):
   df_all.tsv            TERM<TAB>DF           (all distinct stemmed non-stop
      terms, one file)
   top100_df.tsv         TERM<TAB>DF           (100 rows; DF desc, ties by TERM
      asc)
   inverted_index.tsv    TERM<TAB>DOC_ID:TF,DOC_ID:TF,...   (100 rows, postings
      by DOC_ID numeric asc)
   doc_count.txt         N
 Configuration: 1 reducer per job; split.maxsize = 64 MB; Top-K = 100 (-D p2.
      topk=...).

================================================================================
PART 2(b)  -  Query-based retrieval (TF-IDF)
================================================================================
 Inputs  : <results_dir>/inverted_index.tsv  (MapReduce input)
           <results_dir>/top100_df.tsv       (DF values, shipped via
      DistributedCache)
           <results_dir>/doc_count.txt       (N, read by the run script)
           stopwords.txt
 Run (from a folder containing stopwords.txt and opennlp-tools-1.9.3.jar):
   ./run_query.sh <results_dir> "<query string>" [output_dir]
   e.g.  ./run_query.sh results_full "history united states" output_query
 which does:
   javac -cp "$(hadoop classpath):opennlp-tools-1.9.3.jar" -d classes_query
      TextPreprocessor.java QueryRetrieval.java
   jar cf problem2_query.jar -C classes_query .
   export HADOOP_CLASSPATH=$PWD/opennlp-tools-1.9.3.jar
   hadoop jar problem2_query.jar QueryRetrieval -libjars opennlp-tools-1.9.3.jar
       \
       stopwords.txt results_full/top100_df.tsv results_full/inverted_index.tsv
       output_query 10000 "<query>"
 Output  : output_query/part-r-00000, TSV  RANK<TAB>DOC_ID<TAB>MATCHED_TERMS<TAB
      >SCORE
           (top 10, score printed with 6 decimals; all matching docs if fewer
      than 10).

 Definitions
   IDF(t)  = ln( N / DF(t) + 1 )   (natural log; DF(t) read from top100_df.tsv)
   S(D,Q)  = sum over t in Q intersect D of TF(t,D) * IDF(t)
   Q       = SET of stemmed, non-stop query terms: repeated query terms are
      collapsed (counted once).
   Ranking = score descending; ties -> DOC_ID ascending, compared NUMERICALLY.
   Query terms that are stop words vanish in preprocessing; terms not among the
      indexed top-100
   terms are reported on stderr ("Ignored") and skipped. If no usable term
      remains, the program
   prints a message, runs no job and writes no output. If terms are usable but
      no document
   contains them the output file is empty.
   Job counters: MATCHING_DOCS (number of docs scored), DF_MISMATCH (index
      postings count vs DF
```

```
  file; absent = 0 = consistent).

Independent check (no Hadoop; pass the stemmed terms printed by the driver as "
      Terms used for scoring"):

python3 verify_retrieval.py results_full "histori,state,unit" --explain 3
(stdout must equal output_query/part-r-00000; stderr shows DF, IDF and TF x
    IDF per document)
```

6 Limitations

- **Different machines.** The three experiment groups ran on three laptops (Table 1). Timings are comparable only within one experiment; the scalability benchmark (the only timing study) comes entirely from one machine, in standalone mode, so HDFS/YARN effects, network shuffle and multi-node scaling were not measured.
- **Single runs where allowed.** 20% and 80% subsets and all Problem 2 timings are single runs. The 80% Pairs-NoAgg point is slower than the 100% median, which shows the noise level of such single measurements.
- **Non-content tokens.** The agreed tokenizer does not filter numbers, single letters, HTML-entity residue (`apos`, `quot`), URL parts (`http`, `www`) or interlanguage prefixes (`fr`, `pt`...). About 45 of the 100 top-DF terms and several top-50 words are such tokens. The official stop-word list also lacks words such as "this" and "us". We kept the official list and the agreed pipeline unchanged so that all parts stay consistent; filtering would have to be applied to indexing and queries together.
- **Retrieval scope.** 2(b) searches only the 100 highest-DF terms indexed in 2(a); other query terms are ignored, so a query made only of rarer terms returns nothing. This follows the assignment's definition of the index.
- **Ranking quality.** Scoring uses raw TF with no length normalisation, so long, repetitive documents and very common terms dominate (Section 3.4).
- **Query.** No instructor-supplied query was announced; the reported results use our own query. If one is announced, the result files must be regenerated with `run_query.sh`.
- **Stemming.** Porter stems are not always words (`categori`, `histori`); non-ASCII letters act as separators ("café" becomes the token `caf`).
- **Launcher.** The benchmark jobs were launched through Hadoop's `RunJar` with explicit client jars (same for all configurations); the standard `hadoop jar` command runs the same jobs but may have different launcher overhead.